St Francis Institute of Technology

(Autonomous Institute)

Department of Artificial Intelligence and Machine Learning

Second Year AIML Engineering (SEM-IV A.Y. 2025-26)

Web Programming Lab. Experiment Report

Experiment 8: Implement state management in React applications.

---

1. AIM

To implement React State Management using the useState Hook by developing:

- **Part A:** A Login Form with controlled components and dynamic data display
- **Part B:** A Counter Application using useState along with a controlled input form

---

2. LAB OBJECTIVE

- To understand React state and useState
- To implement controlled components
- To manage multiple inputs using a single state object
- To validate user input dynamically
- To build an interactive counter application
- To observe automatic re-rendering when state updates

---

3. LAB OUTCOME

After completing this experiment, students will be able to:

- Implement controlled forms in React
- Manage form data using a single state object
- Perform client-side validation
- Build a counter using useState
- Understand how state updates trigger re-rendering

---

4. PREREQUISITE

- HTML, CSS basics
- JavaScript (functions, objects, events)
- ES6 syntax
- Basic React components
- Props concept

---

5. THEORY

5.1 React State

State is a built-in React object used to store dynamic data in a component.

- When state changes → Component re-renders
- State makes applications interactive
- Managed using Hooks in functional components

• Password strength validation

5.2 useState Hook

Syntax: const [state, setState] = useState(initialValue);

- state → current value
- setState → function to update state
- initialValue → default value

5.3 Controlled Components

A controlled component is an input field controlled by React state.

```
<input value={stateValue} onChange={handleChange} />
```

Benefits:

- Predictable data flow
- Easier validation
- Single source of truth

5.4 Single State Object

Instead of multiple state variables:

```
const [username, setUsername] = useState("");
const [email, setEmail] = useState("");
```

Use:

```
const [formData, setFormData] = useState({
  username: "",
  email: "",
  password: ""
});
```

Advantages:

- Cleaner code
- Scalable structure
- Easier management

5.5 Form Validation

Validation ensures:

- Fields are not empty
- Email format is correct

- Password length is sufficient

Errors are stored in a separate state object and displayed dynamically.

6. PROCEDURE

Part A: Login Form with Controlled Components & Validation

Step 1: Create and Set Up React Application

1. Create a new React application using the appropriate React setup command.
2. Navigate into the project folder.
3. Start the development server.
4. Open the project in a code editor.
5. Clean unnecessary default files (logo, extra CSS, etc.) if required.

Step 2: Create a Login Form Component

1. Create a new functional component named **LoginForm**.
2. Ensure the component is exported properly.
3. Import and render the LoginForm component inside the main App component.

Step 3: Initialize State Using useState

1. Import the useState Hook from React.
2. Create a **single state object** to store:
    - Username
    - Email
    - Password
3. Initialize all fields with empty strings.
4. Create another state object to store validation errors.

Purpose:

- One state object → form data
- One state object → error messages

Step 4: Design the Form Structure

1. Create a form element.
2. Inside the form, create:
    - A text input for Username
    - A text input for Email
    - A password input for Password

    - A Submit button
3. Assign a unique name attribute to each input field.
4. Display validation error messages below each field.

Step 5: Implement Controlled Components

1. Connect each input field's value to the corresponding property in the state object.
2. Create a single change handler function.
3. In the change handler:
    - Capture the input field name and value.
    - Update only the specific field inside the state object.
    - Use the spread operator logic to avoid overwriting other fields.

Outcome:

- Inputs become controlled by React state.
- Any change updates state immediately.

Step 6: Implement Validation Logic

1. Create a separate validation function.
2. Inside the function:
    - Check if username is empty.
    - Check if email is empty.
    - Verify email contains proper format (example: contains "@").
    - Check if password is empty.
    - Ensure password has minimum length (e.g., 6 characters).
3. Store all validation messages inside the error state object.
4. Return the errors from the validation function.

Step 7: Handle Form Submission

1. Create a submit handler function.
2. Prevent default form submission behavior.
3. Call the validation function.
4. If validation errors exist:
    - Update the error state.
    - Stop submission.
5. If no errors:
    - Clear error messages.
    - Display success message.
    - Optionally reset form fields.

Step 8: Display Dynamic User Input

1. Below the form, display:
    - Entered Username
    - Entered Email
2. Bind displayed values directly from the state object.
3. Observe how the data updates automatically when user types.

Step 9: Test the Application

verify:

- Empty form shows required field errors.
- Invalid email shows format error.
- Short password shows length error.
- Valid form displays success message.
- Entered data appears dynamically below the form.
- No page reload occurs during submission.

Part B: Counter App using useState (with Controlled Input)

Step 1: Create the Counter Component

1. Create a new functional component named **Counter**.
2. Export the component properly.
3. Import and render the Counter component inside the main App component.
4. Confirm that the application runs without errors.

Step 2: Import useState Hook

1. Import the useState Hook from React.
2. Understand that useState will allow the component to store and update dynamic values.

Step 3: Initialize State Variables

Create two separate state variables:

1. **Counter State**
    - Initialize with value 0.
    - This will store the current counter value.
2. **Step State**

    - Initialize with value 1.
    - This will control how much the counter increases or decreases.

Purpose:

- count → Stores the current number displayed.
- step → Stores the increment/decrement value entered by user.

Step 4: Design the User Interface

Inside the component:

1. Display a heading such as **"Counter App"**.
2. Create an input field for entering the step value.
3. Display the current counter value.
4. Create three buttons:
    - Increase
    - Decrease
    - Reset

Ensure the layout is clear and readable.

Step 5: Implement Controlled Input for Step Value

1. Connect the step input field to the step state.
2. Make the input field controlled by:
    - Setting its value from state.
    - Updating state whenever user changes the input.
3. Convert input value to a number before storing it in state.
4. Ensure invalid or empty inputs are handled properly.

Outcome:

- Step value updates dynamically.
- Input is fully controlled by React state.

Step 6: Implement Increase Functionality

1. Attach a click event to the **Increase button**.
2. When clicked:
    - Add the current step value to the current counter value.
    - Update the counter state.
3. Observe that the displayed number updates automatically.

Step 7: Implement Decrease Functionality

1. Attach a click event to the **Decrease button**.
2. When clicked:
    - Subtract the step value from the counter value.
    - Update the counter state.
3. Verify that the UI updates immediately.

Step 8: Implement Reset Functionality

1. Attach a click event to the **Reset button**.
2. When clicked:
    - Set the counter value back to 0.
3. Confirm that the display resets correctly.

Step 9: Observe State Re-rendering

Students should observe:

- When counter state changes → Component re-renders.
- When step value changes → Future increments use updated step.
- No page reload occurs.
- UI updates instantly.

Step 10: Test Different Scenarios

Students should test:

- Step value = 1
- Step value = 5
- Step value = negative number
- Step value = 0
- Reset after multiple operations

Verify that:

- Counter behaves correctly.
- Controlled input works as expected.
- State updates are consistent.

---

7. RESULTS / OUTCOME EXPECTED

- How does the counter increment and decrement functionality work in this experiment?
- How is the numeric step input implemented as a controlled component in the counter application?
- What happens in the React component when the counter state changes? Explain the rendering behavior.
- How does the reset button restore the counter to its initial value?

- Explain how multiple state variables are managed efficiently using the useState Hook in this experiment.

---

8. CONCLUSION

- What is the role of useState in this experiment?
- How do controlled components improve form handling?
- Why is a single state object used in the login form?
- How does React know when to re-render the counter?
- What happens if state is updated incorrectly?

---

9. POST-EXPERIMENT QUESTIONS

- Differentiate between props and state.
- What is the advantage of controlled components?
- Why do we use the spread operator while updating state?
- Can multiple useState Hooks be used in one component?
- How can this experiment be extended further?

---

10. REFERENCES

- React Official Documentation – https://react.dev
- MDN JavaScript Documentation
- W3Schools React Tutorial
- *Learning React* – Alex Banks

Question 1 - Solution

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>React State - Login Form</title>
    <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
    <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: 'Segoe UI', sans-serif; background: linear-gradient(135deg, #6...
        h1 { text-align: center; color: white; margin-bottom: 40px; font-size: 2.5em; text...
        .app-container { max-width: 550px; margin: 0 auto; }
        .card { background: white; padding: 40px; border-radius: 20px; box-shadow: 0 20px ...
        .card h2 { color: #667eea; margin-bottom: 25px; font-size: 2em; text-align: center...
        input { width: 100%; padding: 15px; margin: 12px 0; border: 3px solid #ddd; border...
        input:focus { outline: none; border-color: #667eea; box-shadow: 0 0 20px rgba(102,...
        button { background: linear-gradient(135deg, #667eea 0%, #764ba2 100%); color: whi...
        button:hover { transform: translateY(-3px); box-shadow: 0 10px 30px rgba(102, 126,...
        button:active { transform: translateY(0); }
        .welcome { margin-top: 20px; padding: 25px; background: linear-gradient(135deg, #e...
        .welcome p { margin: 10px 0; font-size: 1.1em; color: #2c3e50; }
        .error { color: #ff6b6b; font-size: 0.95em; margin-top: 8px; animation: shake 0.3s...
        @keyframes shake { 0%, 100% { transform: translateX(0); } 25% { transform: transla...
        .logout-btn { background: #ff6b6b; }
        .logout-btn:hover { background: #ee5a5a; box-shadow: 0 10px 30px rgba(255, 107, 10...
    </style>
</head>
<body>
    <div id="root"></div>
    <script type="text/babel">
        const { useState } = React;

        function LoginForm() {
            const [username, setUsername] = useState('');
            const [password, setPassword] = useState('');
            const [loggedIn, setLoggedIn] = useState(false);
            const [error, setError] = useState('');
            const [loginTime, setLoginTime] = useState('');

            const handleLogin = (e) => {
                e.preventDefault();
                if (username.length < 3) {
                    setError('■■ Username must be at least 3 characters');
                    return;
                }
                if (password.length < 6) {
                    setError('■■ Password must be at least 6 characters');
                    return;
                }
                setError('');
                setLoggedIn(true);
                setLoginTime(new Date().toLocaleString());
            };

            const handleLogout = () => {
                setLoggedIn(false);
                setUsername('');
```

Question 2 - Solution

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>React State - Counter & Controlled Input</title>
    <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
    <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: 'Segoe UI', sans-serif; background: linear-gradient(135deg, #1...
        h1 { text-align: center; color: white; margin-bottom: 40px; font-size: 2.5em; text...
        .app-container { max-width: 600px; margin: 0 auto; }
        .card { background: white; padding: 40px; border-radius: 20px; box-shadow: 0 20px ...
        .card h2 { color: #11998e; margin-bottom: 25px; font-size: 2em; text-align: center...
        .card h3 { color: #2c3e50; margin: 20px 0 15px; font-size: 1.4em; }
        input { width: 100%; padding: 15px; margin: 12px 0; border: 3px solid #ddd; border...
        input:focus { outline: none; border-color: #11998e; box-shadow: 0 0 20px rgba(17, ...
        input[type="number"] { width: 100px; display: inline-block; text-align: center; }
        button { background: linear-gradient(135deg, #11998e 0%, #38ef7d 100%); color: whi...
        button:hover { transform: translateY(-3px); box-shadow: 0 10px 30px rgba(17, 153, ...
        button:active { transform: translateY(0); }
        .counter { text-align: center; padding: 20px; }
        .counter .count { font-size: 4.5em; color: #11998e; font-weight: bold; margin: 25p...
        .counter .step-control { margin: 20px 0; }
        .counter .step-control label { font-size: 1.2em; color: #2c3e50; font-weight: 600;...
        .btn-reset { background: #95a5a6; }
        .btn-reset:hover { background: #7f8c8d; box-shadow: 0 10px 30px rgba(149, 165, 166...
        .user-info { margin-top: 20px; padding: 25px; background: linear-gradient(135deg, ...
        .user-info p { margin: 10px 0; font-size: 1.1em; color: #2c3e50; }
        @keyframes slideIn { from { opacity: 0; transform: translateY(-20px); } to { opaci...
        hr { margin: 30px 0; border: none; border-top: 2px solid #f0f2f5; }
    </style>
</head>
<body>
    <div id="root"></div>
    <script type="text/babel">
        const { useState } = React;

        function CounterApp() {
            const [count, setCount] = useState(0);
            const [step, setStep] = useState(1);
            const [name, setName] = useState('');
            const [email, setEmail] = useState('');
            const [submitted, setSubmitted] = useState(false);

            return (
                <div className="app-container">
                    <h1>■■ React State Management</h1>
                    <div className="card">
                        <h2>■ Counter App</h2>
                        <div className="counter">
                            <div className="count">{count}</div>
                            <div className="step-control">
                                <label>Step Size:</label>
                                <input
                                    type="number"
                                    value={step}
                                    onChange={(e) => setStep(Number(e.target.value) || 1)}
```